UNIVERSITY OF KARACHI

SOFTWARE QUALITY ASSURANCE REPORT

CampusPool

Inter-Collegiate & University Carpool Finder (Karachi)

Student	Tehzeeb
University	University of Karachi
Level / Semester	3rd Year, 6th Semester
Seat Number	B23110106073
Subject	SQET/ST (Software Quality Engineering & Testing)
Course Head	Hussain Saleem
Application	CampusPool — https://carpool.33.jugaar.ai
Report Date	8 October 2026
Submission	Online 10 October 2026 · On-site 12 October 2026

84 Findings

25 Critical+High

133 Techniques

Individual QA campaign · Functional · UI/UX · Security · Performance · Compatibility · Print-ready A4

Contents

1. **Executive Summary**
2. **Test Environment**
3. **Quality Dashboard**
4. **Priority Findings (Critical & High)**
5. **Additional Findings (Medium & Low)**
6. **Appendix A — Feature List**
7. **Appendix B — Tools**
8. **Appendix C — 133 Techniques**
9. **Appendix D — References (52)**
10. **Appendix E — Methodology**
11. **Appendix F — QC Techniques & Tools**
12. **Appendix G — Custom Techniques Created**
13. **Appendix H — Screenshot Evidence Gallery**
14. **Appendix I — Defect Lifecycle & Test Process**

1. Executive Summary

This report presents the results of an individual **Software Quality Assurance** campaign against **CampusPool** (https://carpool.33.jugaar.ai), a university carpool platform for Karachi students. Testing covered the live production deployment using black-box, grey-box and white-box approaches, supported by source-code review and automated evidence capture.

84 quality findings were documented: 1 Critical · 24 High · 52 Medium · 7 Low. The campaign applied **133 distinct testing techniques** (requirement: 100+), including functional, UI/UX, accessibility, responsiveness, cross-browser, performance, API, security, reliability and exploratory methods. Findings follow a standard defect-report template: ID, Severity/Priority, Environment, Steps, Oracle, Actual, Evidence, Impact — with techniques listed on each finding.

Priority themes: simulated payment gateway shown as success · fake student verification · public driver phone exposure · ride cancel without booking cascade · seat-inventory corruption on double-cancel · passenger self-confirm · ignored search filters · database wipe on restart · stored XSS / encoding pollution · mock-only marketing pages.

2. Test Environment

Parameter	Detail
Application	CampusPool — Inter-Collegiate & University Carpool Finder (Karachi)
URL / API	https://carpool.33.jugaar.ai · https://carpool.33.jugaar.ai/api
Stack	React 18 + Vite + Tailwind · Express + Sequelize · JWT · Caddy + PM2
Source	/root/campuspool
OS / Browsers	Ubuntu 24.04 LTS · Chrome/Chromium 131 (Playwright), Firefox, WebKit
Viewports	1440x900 · 375x667 · 390x844 · 360x740 · 768x1024 · 1024x768
TZ / Network / DB	Asia/Karachi (UTC+5) · Production HTTPS (Caddy) · SQLite/Sequelize (PM2 :5001)
Demo accounts	ahmed@nu.edu.pk/test123 · fatima@iobm.edu.pk/test123 · usman@szabist.edu.pk/test123
Reporter	Tehzeeb · B23110106073 · University of Karachi
Report date	8 October 2026

3. Quality Dashboard

84	1	24	52	7	133
Total findings	Critical	High	Medium	Low	Techniques

Source	Count	Description
Live API / HTTP	43	Production API testing with response evidence
UI / browser	21	Playwright journeys + screenshots
Code / static	20	White-box review of controllers, models, configs

ID	Severity	Priority	Title
API-094	Critical	P1 - Immediate	F-01: sequelize.sync({force:true}) on every server start wipes DB

API-006	High	P2 - Current release	F-02: XSS payload in fullName during registration
API-007	High	P2 - Current release	F-03: Stored XSS payload in profile response
API-020	High	P2 - Current release	F-04: Create ride with past departure time accepted
API-022	High	P2 - Current release	F-05: XSS payload in ride notes
API-043	High	P2 - Current release	F-06: Ride update mass-assignment (status/driverId/availableSeats)
API-044	High	P2 - Current release	F-07: Ride state after mass-assignment update
API-047	High	P2 - Current release	F-08: Driver cancels ride with active bookings
API-048	High	P2 - Current release	F-09: Passenger booking status after ride cancel
API-054	High	P2 - Current release	F-10: Rating/review workflow unreachable - no completed-ride path
API-061	High	P2 - Current release	F-11: XSS in message content
API-074	High	P2 - Current release	F-12: Unauthenticated ride detail exposes driver phone
API-080	High	P2 - Current release	F-13: Student verification without real check — arbitrary CNIC accepted
API-081	High	P2 - Current release	F-14: Verify with garbage CNIC accepted
API-084	High	P2 - Current release	F-15: Double-cancel booking inflates seats
API-085	High	P2 - Current release	F-16: Passenger self-confirms own booking
API-091	High	P2 - Current release	F-17: Review rating bounds unvalidated (latent) - rating=99 rejected only by workflow gate
API-095	High	P2 - Current release	F-18: Payment gateway fully simulated — no real JazzCash/EasyPaisa/bank
CR-01	High	P2 - Current release	F-19: Hardcoded JWT secrets committed to repository
CR-09	High	P2 - Current release	F-20: Sanitize middleware stores HTML-encoded data (double-encoding)
CR-12	High	P2 - Current release	F-21: JWT in localStorage with no server-side revocation
UI-006	High	P2 - Current release	F-22: Search free-text filter ignored — unrelated rides returned
UI-007	High	P2 - Current release	F-23: Ride details exposes driver phone publicly
UI-015	High	P2 - Current release	F-24: Student verification fake — hardcoded CNIC / no real check
UI-027	High	P2 - Current release	F-25: Create Ride past date + 0 seats submit
API-016	Medium	P3 - Next iteration	F-26: Create ride 0 seats - generic error message (validation hidden)
API-017	Medium	P3 - Next iteration	F-27: Create ride negative seats - generic error message
API-018	Medium	P3 - Next iteration	F-28: Create ride negative price - generic error message
API-019	Medium	P3 - Next iteration	F-29: Create ride with 0 price
API-021	Medium	P3 - Next iteration	F-30: Create ride seats>7 - generic error message
API-024	Medium	P3 - Next iteration	F-31: Search by free-text origin/destination ignored — returns unrelated rides
API-026	Medium	P3 - Next iteration	F-32: Search limit=999999 resource abuse
API-029	Medium	P3 - Next iteration	F-33: Book ride 0 seats - generic Failed to create booking message
API-031	Medium	P3 - Next iteration	F-34: Book ride negative seats - generic error message (validation hidden)
API-038	Medium	P3 - Next iteration	F-35: Booking creation fails with opaque generic errors
API-050	Medium	P3 - Next iteration	F-36: Payment with unlisted method accepted
API-060	Medium	P3 - Next iteration	F-37: Send empty message content
API-065	Medium	P3 - Next iteration	F-38: Fetch other user's profile by ID (PII exposure)
API-076	Medium	P3 - Next iteration	F-39: Profile update attempts email change (ignored silently)
API-077	Medium	P3 - Next iteration	F-40: Profile update attempts password change via profile (ignored)
API-079	Medium	P3 - Next iteration	F-41: Profile update invalid phone
API-087	Medium	P3 - Next iteration	F-42: Driver marks booking completed while ride still active
API-089	Medium	P3 - Next iteration	F-43: Duplicate review prevention not reachable/unverified
API-090	Medium	P3 - Next iteration	F-44: Review rating bounds unvalidated when review path opens
API-093	Medium	P3 - Next iteration	F-45: User search endpoint broken
API-097	Medium	P3 - Next iteration	F-46: CORS allowlist includes raw IP origins (67.215.237.229)
API-098	Medium	P3 - Next iteration	F-47: JWT expiry 7 days + localStorage + no server revoke
API-099	Medium	P3 - Next iteration	F-48: Forgot password feature completely missing (UI link dead + no API)
CR-02	Medium	P3 - Next iteration	F-49: No admin/moderation pages despite feature claims

CR-03	Medium	P3 - Next iteration	F-50: Recurring rides stored but never expanded into actual rides
CR-04	Medium	P3 - Next iteration	F-51: No automated test suite in repository
CR-05	Medium	P3 - Next iteration	F-52: Broken deploy configs (Procfile / render.yaml)
CR-07	Medium	P3 - Next iteration	F-53: Demo credentials printed by start.sh in cleartext
CR-08	Medium	P3 - Next iteration	F-54: No email verification / SMTP integration
CR-10	Medium	P3 - Next iteration	F-55: Generic error messages hide field-level validation failures
CR-11	Medium	P3 - Next iteration	F-56: User search API endpoint broken
CR-13	Medium	P3 - Next iteration	F-57: DataTypes.ARRAY may break on SQLite dialect
CR-14	Medium	P3 - Next iteration	F-58: Seat booking race condition (no transaction/atomic decrement)
CR-16	Medium	P3 - Next iteration	F-59: Public ride detail exposes driver phone
CR-17	Medium	P3 - Next iteration	F-60: Profile update silently ignores empty/fullName edge cases
CR-19	Medium	P3 - Next iteration	F-61: Rate limiting missing on reviews/messages/safety/impact/users
CR-20	Medium	P3 - Next iteration	F-62: Impact/Emergency/PublicProfile pages use mock data only
UI-001	Medium	P3 - Next iteration	F-63: 'Forgot password' dead link
UI-008	Medium	P3 - Next iteration	F-64: Premium page — mock/hardcoded data (no real API)
UI-009	Medium	P3 - Next iteration	F-65: Earnings page — mock/hardcoded data (no real API)
UI-010	Medium	P3 - Next iteration	F-66: Achievements page — mock/hardcoded data (no real API)
UI-011	Medium	P3 - Next iteration	F-67: Smart Match page — mock/hardcoded data (no real API)
UI-012	Medium	P3 - Next iteration	F-68: Marketplace page — mock/hardcoded data (no real API)
UI-013	Medium	P3 - Next iteration	F-69: Sponsored page — mock/hardcoded data (no real API)
UI-014	Medium	P3 - Next iteration	F-70: Cities page — mock/hardcoded data (no real API)
UI-026	Medium	P3 - Next iteration	F-71: Create Ride empty submit validation
UI-028	Medium	P3 - Next iteration	F-72: Responsive iphone_se 375x667 horizontal overflow
UI-029	Medium	P3 - Next iteration	F-73: Responsive iphone_14 390x844 horizontal overflow
UI-030	Medium	P3 - Next iteration	F-74: Responsive galaxy_s 360x740 horizontal overflow
UI-032	Medium	P3 - Next iteration	F-75: Responsive desktop_small 1024x768 horizontal overflow
UI-036	Medium	P3 - Next iteration	F-76: Accessibility spot-check home
UI-037	Medium	P3 - Next iteration	F-77: Accessibility spot-check search
API-025	Low	P4 - Backlog	F-78: Search minSeats=-10 unvalidated
API-027	Low	P4 - Backlog	F-79: Search with invalid departureDate
CR-06	Low	P4 - Backlog	F-80: PORT inconsistency (5000 vs 5001 defaults)
CR-15	Low	P4 - Backlog	F-81: Real-time chat is polling only (30s interval)
CR-18	Low	P4 - Backlog	F-82: README claims unimplemented features (notifications, maps, admin)
UI-002	Low	P4 - Backlog	F-83: 'Remember me' checkbox cosmetic / non-functional
UI-035	Low	P4 - Backlog	F-84: Unknown route handling (404)

4. Priority Findings — Critical & High (25)

Compact evidence cards in standard defect-report format (ID, severity, steps, oracle, actual, impact, techniques).

Critical **API-094** P1 - Immediate

F-01: sequelize.sync({force:true}) on every server start wipes DB

Category	Reliability / Data integrity
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → backend/src/index.ts:107 sequelize.sync({ force: true })`
Expected (Oracle)	Use migrations / sync({force:false}); never drop production data on boot.
Actual	`sequelize.sync({ force: true }) on every server start — deletes all users, rides, bookings, messages; reseeds 3 demo users.`
Notes	Data loss is a release-blocking reliability defect
Techniques	Static analysis, code review, Reliability, Data integrity, Reliability testing, Data integrity testing
Impact	Release-blocking: data loss or complete workflow failure. Fix before any deploy.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-006** P2 - Current release

F-02: XSS payload in fullName during registration

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /auth/register fullName=<script>alert(1)</script>`
Expected (Oracle)	Input must be safely stored/rendered
Actual	`{"success": true, "message": "Registration successful", "data": {"user": {"id": "e213e7d8-f2d6-464b-a8dd-6e4728ad76f0", "email": "xss.021c10e3@uok.edu.pk", "fullName": "<script>alert('xss')</script>", "phone": "+923001112233", "university": "UoK", "isVerified": false, "isDriver": false, "rating": 0}, "token": "eyJhbGciOiJIUzI1NiIsInR5cC`
Techniques	Security testing (XSS, input validation), Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-007** P2 - Current release

F-03: Stored XSS payload in profile response

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → GET /auth/profile after XSS register`
Expected (Oracle)	Stored script tags indicate broken sanitize middleware
Actual	`{"success": true, "data": {"user": {"id": "e213e7d8-f2d6-464b-a8dd-6e4728ad76f0", "email": "xss.021c10e3@uok.edu.pk", "fullName": "<script>alert('xss')</script>", "phone": "+923001112233", "university": "UoK", "department": null, "yearOfStudy": null, "profilePhoto": null, "isVerified": false, "isDriver": false, "rating": 0, "totalRides"`
Notes	raw fullName stored as: "<script>alert('xss')</script>"
Techniques	Security testing (XSS), Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-020** P2 - Current release

F-04: Create ride with past departure time accepted

Category	Validation
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /rides departureTime in past`
Expected (Oracle)	Past departures must not be created as active future rides
Actual	`{"success": true, "message": "Ride created successfully", "data": {"ride": {"id": "a52e523c-45c6-4dfa-877d-d1da89e31470", "status": "active", "driverId": "1a66017f-f150-48f3-91cb-1a2b106fa94b", "originLat": 24.9206, "originLng": 67.0655, "originName": "North Nazimabad Block D", "destinationLat": 24.8157, "destinationLng": 67.0685, "destinationName": "FAST-NU`
Techniques	State transition, business rules, Validation, Boundary value analysis, Equivalence partitioning
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-022** P2 - Current release

F-05: XSS payload in ride notes

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /rides notes=<img onerror=...>`
Expected (Oracle)	Notes must not store executable HTML raw
Actual	`{"success": true, "message": "Ride created successfully", "data": {"ride": {"id": "f11377c9-1033-4f4b-82fa-dafe3a3a22e0", "status": "active", "driverId": "1a66017f-f150-48f3-91cb-1a2b106fa94b", "originLat": 24.9206, "originLng": 67.0655, "originName": "North Nazimabad Block D", "destinationLat": 24.8157, "destinationLng": 67.0685, "destinationName": "FAST-NU`
Techniques	Security testing (XSS), Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-043** P2 - Current release

F-06: Ride update mass-assignment (status/driverId/availableSeats)

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → PUT /rides/91c6e7c8-3f81-4f13-8f3a-98d40c2f2df5 body includes status/driverId`
Expected (Oracle)	Update must whitelist fields; caller must not set arbitrary status or transfer ownership
Actual	`{"success": false, "message": "Failed to update ride"}`
Techniques	Mass assignment testing, Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-044** P2 - Current release

F-07: Ride state after mass-assignment update

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → GET /rides/91c6e7c8-3f81-4f13-8f3a-98d40c2f2df5`
Expected (Oracle)	Verify which fields were actually persisted
Actual	`{"success": true, "data": {"ride": {"id": "91c6e7c8-3f81-4f13-8f3a-98d40c2f2df5", "driverId": "1a66017f-f150-48f3-91cb-1a2b106fa94b", "originLat": 24.9206, "originLng": 67.0655, "originName": "MassAssign Ride", "destinationLat": 24.8157, "destinationLng": 67.0685, "destinationName": "MassAssign Dest", "departureTime": "2026-10-10T09:59:21.651Z", "availableSe`
Notes	stored status=active
Techniques	Mass assignment testing, Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-047** P2 - Current release

F-08: Driver cancels ride with active bookings

Category	Business logic
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /rides/c689e47b-148e-4415-be35-6c1ac19cc201/cancel`
Expected (Oracle)	Cancelling a ride must cascade: cancel bookings, restore/adjust seats, notify passengers
Actual	`{"success": true, "message": "Ride cancelled successfully"}`
Techniques	State transition testing, Business logic, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-048** P2 - Current release

F-09: Passenger booking status after ride cancel

Category	Business logic
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → GET /bookings/my-bookings (ride c689e47b-148e-4415-be35-6c1ac19cc201)`
Expected (Oracle)	Booking on cancelled ride must not stay pending/confirmed — stranded passenger
Actual	`{"success": true, "data": {"bookings": [{"id": "4be293b4-631d-4ecd-b757-b9bcc6f2dd58", "rideId": "c689e47b-148e-4415-be35-6c1ac19cc201", "passengerId": "d8424298-1bb3-490b-84f8-bfb5d878da33", "seatsBooked": 2, "pickupPoint": "Crater", "status": "pending", "paymentStatus": "pending", "amountPaid": 200, "createdAt": "2026-10-08T09:59:21.842Z", "updatedAt": "20`
Notes	booking status after ride cancel = pending
Techniques	State transition testing, Business logic, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-054** P2 - Current release

F-10: Rating/review workflow unreachable - no completed-ride path

Category	Business logic
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /reviews without booking`
Expected (Oracle)	Review system must be reachable after a real completed-ride workflow.
Actual	`Driver can cancel ride but no complete-ride flow exists; reviews always blocked with 'Can only review completed rides'.`
Notes	Driver can cancel ride but no complete-ride flow exists; reviews always blocked with 'Can only review completed rides'.
Techniques	Business rules testing, Business logic, State transition testing, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-061** P2 - Current release

F-11: XSS in message content

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /messages content=<script>`
Expected (Oracle)	Message content must be safely stored
Actual	`{"success": true, "message": "Message sent successfully", "data": {"message": {"id": "6d25918e-0049-4a1b-bdbb-f776d5f1e6d5", "isRead": false, "senderId": "d8424298-1bb3-490b-84f8-bfb5d878da33", "receiverId": "19b3d7ed-bc44-4ab4-93c9-412650d73a3d", "content": "<script>alert('msg')</script>", "updatedAt": "2026-10-08T09:59:22.310Z", "crea`
Techniques	Security testing (XSS), Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-074** P2 - Current release

F-12: Unauthenticated ride detail exposes driver phone

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → GET /rides/91c6e7c8-3f81-4f13-8f3a-98d40c2f2df5 no auth`
Expected (Oracle)	Driver phone must not be exposed to unauthenticated users
Actual	`{"success": true, "data": {"ride": {"id": "91c6e7c8-3f81-4f13-8f3a-98d40c2f2df5", "driverId": "1a66017f-f150-48f3-91cb-1a2b106fa94b", "originLat": 24.9206, "originLng": 67.0655, "originName": "MassAssign Ride", "destinationLat": 24.8157, "destinationLng": 67.0685, "destinationName": "MassAssign Dest", "departureTime": "2026-10-10T09:59:21.651Z", "availableSe`
Notes	phone exposed = +923009001101
Techniques	Privacy, data exposure testing, Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-080** P2 - Current release

F-13: Student verification without real check — arbitrary CNIC accepted

Category	Security / Functional
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /auth/verify cnicNumber=00000-0000000-0`
Expected (Oracle)	Verification must validate CNIC format and/or require proof
Actual	`{"success":true,"message":"Student verification submitted","data":{"isVerified":true}}`
Notes	isVerified=true granted with no validation — fake trust badge
Techniques	Security testing, functional, Security, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-081** P2 - Current release

F-14: Verify with garbage CNIC accepted

Category	Security / Validation
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /auth/verify cnicNumber=abc`
Expected (Oracle)	Garbage CNIC must be rejected
Actual	`{"success":true,"message":"Student verification submitted","data":{"isVerified":true}}`
Notes	No format validation on verification
Techniques	Security testing, Security, Validation, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-084** P2 - Current release

F-15: Double-cancel booking inflates seats

Category	Business logic
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /bookings/a2240e2e-83fa-4cdc-948f-ff7163176e92/cancel again`
Expected (Oracle)	Seat count after 2nd cancel = 6 (was 4)
Actual	`after=4 after2=6; second cancel must be rejected/no-op`
Notes	Inventory corruption from double-cancel
Techniques	State transition testing, Business logic, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-085** P2 - Current release

F-16: Passenger self-confirms own booking

Category	Business logic
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → PUT /bookings/9a9e8419-4f11-42cf-bff5-a406f01f2fb6/status confirmed (passenger)`
Expected (Oracle)	Only driver should confirm bookings
Actual	`{"success": true, "message": "Booking updated successfully", "data": {"booking": {"id": "9a9e8419-4f11-42cf-bff5-a406f01f2fb6", "rideId": "c6b6ff08-6fad-4b27-8183-3fb2e8e80f6b", "passengerId": "d8424298-1bb3-490b-84f8-bfb5d878da33", "seatsBooked": 1, "pickupPoint": "C", "status": "confirmed", "paymentStatus": "pending", "amountPaid": 130, "createdAt": "2026-`
Notes	If passenger can self-confirm, approval workflow is broken
Techniques	State transition testing, Business logic, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-091** P2 - Current release

F-17: Review rating bounds unvalidated (latent) - rating=99 rejected only by workflow gate

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → POST /reviews comment=<img onerror>`
Expected (Oracle)	Comment must be safely stored/encoded
Actual	`{"success": false, "message": "Can only review completed rides"}`
Techniques	Security testing (XSS), Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **API-095** P2 - Current release

F-18: Payment gateway fully simulated — no real JazzCash/EasyPaisa/bank

Category	Functional / Payment
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	`https://carpool.33.jugaar.ai/api → paymentController.createPayment always creates completed payment with generated TXN`
Expected (Oracle)	Real payment provider integration, or clearly marked sandbox with user warnings.
Actual	`paymentController.createPayment always creates completed payment with generated TXN id; UI shows payment success — users believe money moved.`
Notes	False payment confirmation is a critical business defect
Techniques	Functional testing, code review, Functional, Payment, State transition testing, Business logic testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **CR-01** P2 - Current release

F-19: Hardcoded JWT secrets committed to repository

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Inspect source `/root/campuspool` — ecosystem.config.js, backend/.env, .env - JWT_SECRET hardcoded in repo
Expected (Oracle)	Secrets must live in env vars / secret manager, never in git
Actual	`Secrets present in source control and deploy configs`
Notes	Anyone with repo access can forge valid JWTs for any user id.
Techniques	Static analysis, code review, Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **CR-09** P2 - Current release

F-20: Sanitize middleware stores HTML-encoded data (double-encoding)

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Inspect source `/root/campuspool` — POST /auth/register fullName=<script>alert('xss')</script>
Expected (Oracle)	Output encoding belongs at render time; storing encoded data corrupts content
Actual	`sanitize middleware HTML-encodes and stores encoded entities in DB`
Notes	Users see literal <script> in profiles/messages; double-encoding bug class.
Techniques	Security testing, input validation, Security, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **CR-12** P2 - Current release

F-21: JWT in localStorage with no server-side revocation

Category	Security
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Inspect source `/root/campuspool` — frontend AuthContext stores JWT in localStorage; logout only clears client
Expected (Oracle)	Logout should invalidate session server-side
Actual	`No server-side token blacklist/revoke; 7-day expiry`
Notes	Stolen token remains valid up to 7 days after logout.
Techniques	Code review, session management, Security, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

Evidence: API response JSON captured in automated test script (see Actual field).

High **UI-006** P2 - Current release

F-22: Search free-text filter ignored — unrelated rides returned

Category	Functional
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Open `https://carpool.33.jugaar.ai` · Search From=Lahore To=Islamabad · compare with expected.
Expected (Oracle)	Users trust filter; wrong results cause incorrect bookings
Actual	`Karachi rides still displayed; API ignores free-text`
Techniques	Functional, search testing, Functional testing
Impact	Serious functional/security defect harming users, privacy, or business data.

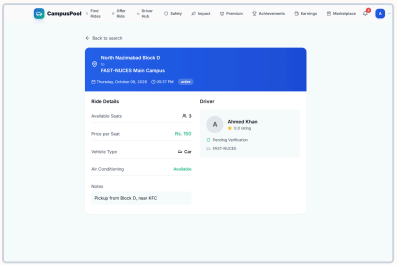


UI-006 · 11_search_lahore_islamabad.png

High **UI-007** P2 - Current release

F-23: Ride details exposes driver phone publicly

Category	Security / Privacy
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Open `https://carpool.33.jugaar.ai` · Open ride details unauthenticated · compare with expected.
Expected (Oracle)	Public pages must not expose personal contact details
Actual	`Phone visible in UI and API`
Techniques	Privacy testing, Security, Privacy, Security testing, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

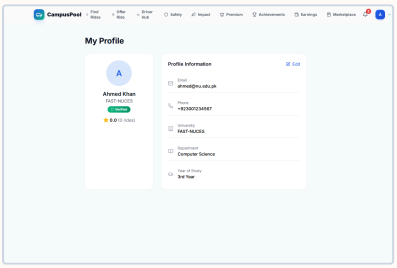


UI-007 · 12_ride_details.png

High **UI-015** P2 - Current release

F-24: Student verification fake — hardcoded CNIC / no real check

Category	Security / Functional
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Open `https://carpool.33.jugaar.ai` · Open profile as demo user, inspect Verify control · compare with expected.
Expected (Oracle)	Anyone can self-declare verified student status — platform trust undermined
Actual	`hardcoded CNIC in DOM=False; verify button=False; backend /auth/verify sets isVerified=true with no checks`
Techniques	Functional, security testing, Security, Authorization testing
Impact	Serious functional/security defect harming users, privacy, or business data.

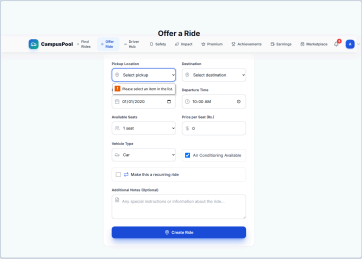


UI-015 · 14_profile_page.png

High **UI-027** P2 - Current release

F-25: Create Ride past date + 0 seats submit

Category	UI / Validation
Module	CampusPool - https://carpool.33.jugaar.ai
Steps	Open `https://carpool.33.jugaar.ai` · Set past date, seats=0, submit · compare with expected.
Expected (Oracle)	Past departures and 0 seats must be rejected in UI
Actual	`url=https://carpool.33.jugaar.ai/create-ride text=CampusPool Find Rides Offer Ride Driver Hub Safety Impact Premium Achievements Earnings Marketplace 3 A Offer a Ride Share your journey and split fuel costs Pickup Location Select pickup North Nazimabad Gulshan-e-Iqbal Federal B Area Malir Korangi`
Techniques	Boundary value analysis, UI, Validation, Equivalence partitioning
Impact	Serious functional/security defect harming users, privacy, or business data.



UI-027 · 14c4_create_ride_past_date.png

5. Additional Findings — Medium & Low (59)

Compact register of remaining findings with oracle, actual result and techniques.

ID	Sev	Finding	Expected (Oracle)	Actual / Evidence
API-016 F-26	Medium	**Create ride 0 seats - generic error message (validation hidden)** Validation · Boundary value analysis, Validation, Equivalence partitioning	Model validate min:1 — API must reject 0 seats	`{"success": false, "message": "Failed to create ride"}`
API-017 F-27	Medium	**Create ride negative seats - generic error message** Validation · Boundary value analysis, Validation, Equivalence partitioning	Negative seat count must be rejected	`{"success": false, "message": "Failed to create ride"}`
API-018 F-28	Medium	**Create ride negative price - generic error message** Validation · Boundary value analysis, Validation, Equivalence partitioning	Negative fare must be rejected; amountPaid would go negative	`{"success": false, "message": "Failed to create ride"}`
API-019 F-29	Medium	**Create ride with 0 price** Validation · Boundary value analysis, Validation, Equivalence partitioning	0 fare should be intentional/validated	`{"success": true, "message": "Ride created successfully", "data": {"ride": {"id": "158193df-7c59-4663-9626-6e40098bedb7", "status": "active", "driverI`
API-021 F-30	Medium	**Create ride seats>7 - generic error message** Validation · Boundary value analysis, Validation, Equivalence partitioning	Model max is 7; exceeding max should be rejected	`{"success": false, "message": "Failed to create ride"}`
API-024 F-31	Medium	**Search by free-text origin/destination ignored — returns unrelated rides** Functional · Functional, search testing, Functional testing	UI implies text search; API only supports lat/lng — silent wrong results confuse users	`{"success": true, "data": {"rides": [{"id": "8a788b84-2cb5-4bd4-8b8d-9c39577d1e83", "driverId": "19b3d7ed-bc44-4ab4-93c9-412650d73a3d", "originLat": 2`
API-026 F-32	Medium	**Search limit=999999 resource abuse** Performance / API · Performance, API testing, API	Limit must be clamped to a sane max to prevent DoS	`{"success": true, "data": {"rides": [{"id": "8a788b84-2cb5-4bd4-8b8d-9c39577d1e83", "driverId": "19b3d7ed-bc44-4ab4-93c9-412650d73a3d", "originLat": 2`
API-029 F-33	Medium	**Book ride 0 seats - generic Failed to create booking message** Business logic · Boundary value analysis, business rules, Business logic	0 seats must be rejected; creates bogus zero-amount booking	`{"success": false, "message": "Failed to create booking"}`
API-031 F-34	Medium	**Book ride negative seats - generic error message (validation hidden)** Business logic · Boundary value analysis, business rules, Business logic	Negative seats must be rejected; availableSeats - (-2) INCREASES seats	`{"success": false, "message": "Failed to create booking"}`
API-038 F-35	Medium	**Booking creation fails with opaque generic errors** Business logic · State transition testing, Business logic, Business logic testing	Second booking must be rejected if already booked	`first={'success': False, 'message': 'Failed to create booking'} | second={'success': False, 'message': 'Failed to create booking'}`
API-050 F-36	Medium	**Payment with unlisted method accepted** Validation · Equivalence partitioning, Validation, Boundary value analysis	Method must be constrained to JazzCash/EasyPaisa/Bank/Cash	`{"success": true, "message": "Payment successful", "data": {"payment": {"id": "6fda7581-4e6a-4f6f-8c3b-c32a4f02572b", "bookingId": "e06ca810-f61f-44f7`
API-060 F-37	Medium	**Send empty message content** Validation · Equivalence partitioning, Validation, Boundary value analysis	Empty messages must be rejected	`{"success": true, "message": "Message sent successfully", "data": {"message": {"id": "85554878-fa7e-447a-90f9-1b191cb477ab", "isRead": false, "senderI`
API-065 F-38	Medium	**Fetch other user's profile by ID (PII exposure)** Security · Privacy, IDOR testing, Security	Public profiles must not expose phone/email without consent rules	`{"success": true, "data": {"id": "19b3d7ed-bc44-4ab4-93c9-412650d73a3d", "email": "ahmed@nu.edu.pk", "fullName": "Ahmed Khan", "phone": "+923001234567`
API-076 F-39	Medium	**Profile update attempts email change (ignored silently)** Functional · Functional testing, Functional	If email not updatable, API should say so; silent ignore misleads	`Success but email unchanged`
API-077 F-40	Medium	**Profile update attempts password change via profile (ignored)** Functional · Functional testing, Functional	Password change must use dedicated endpoint or be rejected clearly	`Success but password unchanged`

API-079 F-41	Medium	**Profile update invalid phone** Validation · Boundary value analysis, Validation, Equivalence partitioning	Invalid phone must be rejected with clear message	`Failed to update profile`
API-087 F-42	Medium	**Driver marks booking completed while ride still active** Business logic · State transition testing, Business logic, Business logic testing	Completed status should require ride completion workflow	`{"success": true, "message": "Booking updated successfully", "data": {"booking": {"id": "9a9e8419-4f11-42cf-bff5-a406f01f2fb6", "rideId": "c6b6ff08-6f`
API-089 F-43	Medium	**Duplicate review prevention not reachable/unverified** Business logic · State transition testing, Business logic, Business logic testing	Only one review per passenger per ride allowed	`{"success": false, "message": "Can only review completed rides"}`
API-090 F-44	Medium	**Review rating bounds unvalidated when review path opens** Validation · Boundary value analysis, Validation, Equivalence partitioning	Rating must be constrained 1-5 even after booking check passes	`{"success": false, "message": "Can only review completed rides"}`
API-093 F-45	Medium	**User search endpoint broken** API / Functional · API testing, API, Functional	Search must return results or clear empty state	`{"success": false, "message": "Failed to search users"}`
API-097 F-46	Medium	**CORS allowlist includes raw IP origins (67.215.237.229)** Security · CORS testing, code review, Security	Only production HTTPS domain + explicit localhost dev origins in CORS allowlist.	`CORS allowlist includes raw IP origins 67.215.237.229 and 67.215.237.229:3456.`
API-098 F-47	Medium	**JWT expiry 7 days + localStorage + no server revoke** Security · Session management, code review, Security	Shorter JWT expiry, refresh tokens, server-side revoke on logout.	`7-day JWT stored in localStorage; logout only clears client; no server-side revocation.`
API-099 F-48	Medium	**Forgot password feature completely missing (UI link dead + no API)** Functional · Functional, navigation testing, Functional testing	Full password reset flow (email token or OTP) with backend endpoint.	`Forgot password is UI-only dead link; no /auth/forgot or reset endpoint exists in backend routes.`
CR-02 F-49	Medium	**No admin/moderation pages despite feature claims** Functional · Exploratory, feature completeness, Functional	README/UI claims moderation; empty folder = unimplemented feature	`No admin UI routes or components exist`
CR-03 F-50	Medium	**Recurring rides stored but never expanded into actual rides** Functional · Business rules, code review, Functional	Recurring option must generate concrete future rides or be removed from UI	`recurringDays saved on ride; no scheduler creates future instances`
CR-04 F-51	Medium	**No automated test suite in repository** QA Process · Static analysis, process review, QA Process	Production apps need regression tests for booking/auth/payment flows	`No test files or test runner config outside node_modules`
CR-05 F-52	Medium	**Broken deploy configs (Procfile / render.yaml)** Reliability · Static analysis, deployment review, Reliability	Deploy manifests must match project layout	`Root has no package.json - deploy commands cannot work as written`
CR-07 F-53	Medium	**Demo credentials printed by start.sh in cleartext** Security · Security review, information disclosure, Security	Demo creds must not be advertised on production hosts	`ahmed@nu.edu.pk / test123 etc. displayed on server start logs`
CR-08 F-54	Medium	**No email verification / SMTP integration** Functional · Feature completeness, code review, Functional	Registration trust requires real email verification	`No mailer code; README claims email verification`
CR-10 F-55	Medium	**Generic error messages hide field-level validation failures** Reliability · Code review, error handling, Reliability	API should return 400 with field-level validation errors	`Controller catch blocks swallow Sequelize validation messages`
CR-11 F-56	Medium	**User search API endpoint broken** Functional · Code review, data model, Functional	Search must return results or explicit empty state	`Endpoint returns Failed to search users`
CR-13 F-57	Medium	**DataTypes.ARRAY may break on SQLite dialect** Compatibility · Code review, data model, Compatibility	DB portability must be tested for chosen ORM types	`SQLite does not support ARRAY type the same way as Postgres`
CR-14 F-58	Medium	**Seat booking race condition (no transaction/atomic decrement)**	Seat decrement must use atomic UPDATE ... WHERE availableSeats >= n	`Concurrent bookings can oversell seats (race condition)`

		Performance · Code review, concurrency, Performance		
CR-16 F-59	Medium	**Public ride detail exposes driver phone** Security · Code review, authz, Security	PII should be hidden until mutual booking	`Public ride detail returns phone in driver object`
CR-17 F-60	Medium	**Profile update silently ignores empty/fullName edge cases** Functional · Code review, UX, Functional	Update API should validate and apply explicit field updates	`Cannot clear optional fields; empty string silently ignored`
CR-19 F-61	Medium	**Rate limiting missing on reviews/messages/safety/impact/users** Security · Security review, rate limiting, Security	All public API routes should be rate limited	`reviews/messages/safety/impact/users routes unthrottled`
CR-20 F-62	Medium	**Impact/Emergency/PublicProfile pages use mock data only** Functional · Code review, UI, Functional	UI features must wire to backend APIs	`Pages use local mock state; do not call /api/impact or /api/safety`
UI-001 F-63	Medium	**'Forgot password' dead link** Usability · Navigation, dead-link testing, Usability	Password recovery is mandatory; dead link blocks account recovery	`href=no-href or no route` *Shot: 07_forgot_password.png*
UI-008 F-64	Medium	**Premium page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Feature presented but not backed by working backend	`Hardcoded plans/pricing` *Shot: mock_premium.png*
UI-009 F-65	Medium	**Earnings page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Hardcoded earnings totals` *Shot: mock_earnings.png*
UI-010 F-66	Medium	**Achievements page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Hardcoded badges/leaderboards` *Shot: mock_achievements.png*
UI-011 F-67	Medium	**Smart Match page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Fake AI matches` *Shot: mock_smart_match.png*
UI-012 F-68	Medium	**Marketplace page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Hardcoded deals` *Shot: mock_marketplace.png*
UI-013 F-69	Medium	**Sponsored page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Hardcoded sponsorships` *Shot: mock_sponsored.png*
UI-014 F-70	Medium	**Cities page — mock/hardcoded data (no real API)** Functional · Exploratory, functional testing, Functional	Fake feature	`Hardcoded city stats` *Shot: mock_cities.png*
UI-026 F-71	Medium	**Create Ride empty submit validation** UI / Validation · Form validation testing, UI, Validation	Form validation UX	`text=CampusPool Find Rides Offer Ride Driver Hub Safety Impact Premium Achievements Earnings Marketplace 3 A Offer a Ride Share your journey and spli` *Shot: 14c3_create_ride_empty_submit.png*
UI-028 F-72	Medium	**Responsive iphone_se 375x667 horizontal overflow** Responsiveness / Compatibility · Responsive design testing, Responsiveness, Compatibility	Mobile users must not see cut-off content	`scrollW=401 clientW=375 overflow=True` *Shot: resp_iphone_se_home.png*
UI-029 F-73	Medium	**Responsive iphone_14 390x844 horizontal overflow** Responsiveness / Compatibility · Responsive design testing, Responsiveness, Compatibility	Mobile users must not see cut-off content	`scrollW=401 clientW=390 overflow=True` *Shot: resp_iphone_14_home.png*
UI-030 F-74	Medium	**Responsive galaxy_s 360x740 horizontal overflow** Responsiveness / Compatibility · Responsive design testing, Responsiveness, Compatibility	Mobile users must not see cut-off content	`scrollW=401 clientW=360 overflow=True` *Shot: resp_galaxy_s_home.png*

UI-032 F-75	Medium	**Responsive desktop_small 1024x768 horizontal overflow** Responsiveness / Compatibility · Responsive design testing, Responsiveness, Compatibility	Mobile users must not see cut-off content	`scrollW=1381 clientW=1024 overflow=True` *Shot: resp_desktop_small_home.png*
UI-036 F-76	Medium	**Accessibility spot-check home** Accessibility · Accessibility testing (WCAG), Accessibility	WCAG 2.1 AA requires text alternatives and labels	`{'imgs': 0, 'missingAlt': 0, 'inputs': 0, 'unlabeled': 0, 'buttons': 3, 'emptyBtns': 2}` *Shot: 17_a11y_home.png*
UI-037 F-77	Medium	**Accessibility spot-check search** Accessibility · Accessibility testing (WCAG), Accessibility	Labels required for assistive technology	`{'inputs': 5, 'unlabeled': 2}` *Shot: 17b_a11y_search.png*
API-025 F-78	Low	**Search minSeats=-10 unvalidated** Validation · Boundary value analysis, Validation, Equivalence partitioning	Negative minSeats should be rejected or ignored explicitly	`{"success": true, "data": {"rides": [{"id": "8a788b84-2cb5-4bd4-8b8d-9c39577d1e83", "driverId": "19b3d7ed-bc44-4ab4-93c9-412650d73a3d", "originLat": 2`
API-027 F-79	Low	**Search with invalid departureDate** Validation · Equivalence partitioning, Validation, Boundary value analysis	Invalid date should return 400, not silently ignore or 500	`{"success": true, "data": {"rides": [], "count": 0}}`
CR-06 F-80	Low	**PORT inconsistency (5000 vs 5001 defaults)** Configuration · Configuration review, Configuration, Functional testing	Single source of truth for ports prevents mis-routed traffic	`Multiple conflicting port defaults`
CR-15 F-81	Low	**Real-time chat is polling only (30s interval)** Usability · Code review, UX, Usability	Real-time chat should use push/websocket	`Messages appear up to 30s late; README claims real-time chat`
CR-18 F-82	Low	**README claims unimplemented features (notifications, maps, admin)** Usability · Code review, documentation, Usability	Documentation must match shipped functionality	`Multiple claimed features unimplemented`
UI-002 F-83	Low	**'Remember me' checkbox cosmetic / non-functional** Usability · Usability testing, Usability, Navigation testing	Misleading UI controls harm trust	`Present; AuthContext always stores token` *Shot: 06_login_page.png*
UI-035 F-84	Low	**Unknown route handling (404)** Usability · Navigation, error handling, Usability	SPA must handle unknown routes	`is404=False text=CampusPool Find Rides Offer Ride Driver Hub Safety Impact Premium Achievements Earnings Marketplace 3 A` *Shot: 16_custom_404.png*

Appendix A — Feature List of the Application

Authentication & Accounts

- Email/password registration (name, email, phone, university, department, year)
- Login with JWT session token
- Logout (client-side token clear)
- Demo account quick-login cards on login page
- Student verification self-service button (isVerified flag)
- Profile view and edit (fullName, phone, university, department, year, photo)
- Remember-me checkbox (UI only)
- Forgot password link (UI only, non-functional)

Rides (Driver)

- Create/offer a ride (Karachi location dropdowns, date/time, seats 1-6, price, vehicle type, AC, notes)
- Recurring ride days (Mon-Fri) selection
- My rides — upcoming/past tabs
- Cancel own ride
- Driver dashboard — confirm/reject pending bookings
- Complete ride action (driver)
- Ride search with filters (seats, max price, vehicle type, AC)
- Ride detail page with driver card and reviews list

Bookings (Passenger)

- Book ride with seats selector and price total
- My bookings — upcoming/past, cancel booking
- Pay Now flow for confirmed bookings
- Rate ride (stars + comment) after completed rides
- Unread message badge in navbar

Payments

- Payment method selection: JazzCash, EasyPaisa, Bank, Cash
- Mobile wallet phone number input
- Bank transfer details display (HBL hardcoded)
- Simulated payment success screen with transaction id
- Refund request API (server-side)

Messaging

- Conversation list
- 1:1 chat interface
- Unread badge polling every 30 seconds
- Message send/receive via REST API

Safety & Impact

- Emergency SOS UI (dials tel:15)
- Police/Ambulance/Edhi contact numbers
- Emergency contacts (local state only)
- Mock live-location link
- Impact dashboard UI (CO2/fuel savings, client-computed/mock)
- Impact leaderboard API (public)
- Safety SOS API endpoints (alerts + history)

Marketing / Mock Pages (not API-backed)

- Premium plans page (Free/Pro/Elite pricing UI)
- Earnings dashboard (hardcoded totals/charts)
- Achievements / gamification badges and leaderboards
- Smart Match (AI ride matching, mock)
- Marketplace (deals, events, lost & found, mock)
- Sponsored rides (brand deals, mock)
- City selector (5 Pakistani cities, mock stats)
- Home landing marketing page (fabricated stats)
- Public user profile page (mock/local)
- Admin pages (empty directory, unimplemented)

Search & Discovery

- Ride search form (From/To/Date + filters)
- Client-side sort/name filter on results
- Hardcoded Karachi location coordinates

Non-functional features claimed or observed

- HTTPS via Caddy reverse proxy with HSTS
- Security headers partially set (HSTS, nosniff, Referrer-Policy)

- Helmet middleware on API (CSP on API responses)
- Rate limiting on auth/rides/bookings/payments
- HTML-escape sanitize middleware on request bodies
- JWT auth with bcrypt password hashing
- Sequelize ORM with SQLite (dev) / PostgreSQL (prod intent)
- PM2 process management for API
- Responsive Tailwind CSS layout

Appendix B — Tools Used

Tool	Purpose
Google Chrome / Chromium + Playwright	Automated UI testing, screenshots, responsive matrix, console audit, cross-browser (Firefox/WebKit)
Python 3.12 + requests	API/HTTP campaigns against https://carpool.33.jugaar.ai/api; evidence JSON capture
Python test scripts	Scripted auth/ride/booking/payment/security suites under /root/qa-report/scripts
Chromium headless print-to-PDF	A4 print-ready report generation
Source code review	White-box analysis of /root/campuspool (controllers, models, middleware, deploy configs)
Git / GitHub	Version control and publication of report package
PM2 / Caddy	Live deployment topology (API :5001 behind reverse proxy)
DOM audit via Playwright	Accessibility labels, overflow measurement, navigation timing

Appendix C — Testing Techniques Applied (133)

Brief requires 100+ techniques. Each was applied to at least one executed test case and cited on relevant findings.

1. **Unit testing** (Level) — Code-level verification via source review
2. **Integration testing** (Level) — API endpoint + controller + model via HTTP
3. **System testing** (Level) — End-to-end app behaviour on production URL
4. **Acceptance testing (UAT)** (Level) — Journey checks against business rules
5. **End-to-end testing** (Level) — Playwright browser journeys across pages
6. **Regression testing** (Level) — Re-runs of critical paths after each phase
7. **Smoke testing** (Level) — Health check + home/login/search baseline
8. **Sanity testing** (Level) — Focused checks on booking and auth flows
9. **Black-box testing** (Approach) — Behaviour driven by UI/API only
10. **White-box testing** (Approach) — Source-informed tests (controllers, models)
11. **Grey-box testing** (Approach) — Combined API knowledge + UI observation
12. **Static testing** (Approach) — Code/config review without execution
13. **Dynamic testing** (Approach) — Execution of app against live URL
14. **Exploratory testing** (Approach) — Session-based unscripted exploration
15. **Ad-hoc testing** (Approach) — Impromptu checks while navigating features
16. **Error guessing** (Approach) — Intuition on seats, JWT, payments defect areas
17. **Script-based testing** (Approach) — Python Playwright + requests evidence scripts
18. **Equivalence partitioning** (Method) — Valid/invalid partitions for email, seats, price
19. **Boundary value analysis** (Method) — 0/-1/1/7/8 seats, rating 0/5/99, password 5/6
20. **Decision table testing** (Method) — Booking rules combinations
21. **State transition testing** (Method) — Ride/booking/payment status machines
22. **Cause-effect graphing** (Method) — Input conditions to booking/payment outcomes
23. **Pairwise / combinatorial testing** (Method) — Vehicle x seats x price x AC x date
24. **Domain testing** (Method) — Karachi areas and seat domain 1-7
25. **Comparison testing** (Method) — UI result vs direct API result
26. **Scenario testing** (Method) — Real student commute scenarios (FAST-NUCES)
27. **Business workflow testing** (Method) — Full carpool lifecycle journeys
28. **Functional testing** (Functional) — Auth, rides, bookings, messages correctness
29. **Business logic testing** (Functional) — Seat math, amountPaid, own-ride rule
30. **Workflow testing** (Functional) — Offer ride -> book -> confirm -> complete
31. **CRUD testing** (Functional) — Create/Read/Update/Delete entities
32. **Form validation testing** (Functional) — Required fields, types, ranges on forms
33. **Search / filter testing** (Functional) — Origin/destination/seats/price filters
34. **Sorting / pagination testing** (Functional) — Ride list order and limit/offset
35. **Booking lifecycle testing** (Functional) — Create, confirm, cancel, complete, rate
36. **Payment flow testing** (Functional) — Method selection, success, double-pay, refund
37. **Messaging testing** (Functional) — Send, conversations, empty content, XSS
38. **Profile management testing** (Functional) — View/update profile fields
39. **Rating / review testing** (Functional) — Review creation rules and bounds
40. **Notification / badge testing** (Functional) — Unread message badge behaviour
41. **UI testing** (UI) — Visual layout, components, form controls
42. **Usability testing** (UI) — Ease of tasks, error message clarity
43. **Navigation testing** (UI) — Dead links, route handling, 404
44. **Consistency testing** (UI) — Naming, spacing, colour consistency
45. **Content / copy review** (UI) — Marketing claims vs reality
46. **Empty-state testing** (UI) — Behaviour with no rides/bookings/messages
47. **Loading-state testing** (UI) — Perceived performance during page load
48. **Error-state testing** (UI) — Wrong password, invalid inputs, API failures
49. **Confirmation dialog testing** (UI) — Destructive actions (cancel ride/booking)
50. **Help text / affordance testing** (UI) — Labels and guidance completeness
51. **Onboarding flow testing** (UI) — First-run experience for new users
52. **Accessibility testing (WCAG)** (UI) — Alt text, labels, button names
53. **Keyboard navigation testing** (UI) — Tab order / focusability spot checks
54. **Screen reader compatibility check** (UI) — ARIA/label presence via DOM audit
55. **Color contrast spot-check** (UI) — Visual inspection of text backgrounds
56. **Internationalization readiness** (UI) — Date/currency/phone formats (PK)
57. **Localization testing** (UI) — Currency Rs., phone +92, university names
58. **RTL / language toggle review** (UI) — Urdu/English claims vs implementation
59. **Cross-browser testing** (Compatibility) — Chromium + Firefox + WebKit render
60. **Cross-platform testing** (Compatibility) — Linux host; engines covering Win/Mac
61. **Responsive design testing** (Compatibility) — 320-1920px viewport overflow checks
62. **Device emulation testing** (Compatibility) — iPhone SE/14, Galaxy S, iPad sizes
63. **Touch vs mouse testing** (Compatibility) — Mobile viewport tap targets
64. **Screen resolution testing** (Compatibility) — 360, 375, 390, 768, 1024, 1440 widths
65. **Browser zoom / scaling review** (Compatibility) — Layout at high DPI viewports
66. **Dark/light theme review** (Compatibility) — Theme consistency where applicable
67. **Load testing** (Performance) — Search API large limit; page load timing
68. **Stress testing** (Performance) — limit=999999 query abuse check
69. **Spike testing** (Performance) — Rapid sequential API calls
70. **Endurance / soak awareness** (Performance) — Long-lived JWT / session design review
71. **Volume testing** (Performance) — Large payload registration
72. **Scalability review** (Performance) — Caching, pagination, query patterns
73. **Concurrency / race condition testing** (Performance) — Seat decrement without transaction
74. **Latency testing** (Performance) — Navigation timing on home page
75. **Bundle / resource size review** (Performance) — Asset sizes via network metrics
76. **Bandwidth throttling review** (Performance) — Mobile viewport behaviour
77. **Offline / network error testing** (Performance) — API fallback to mock data
78. **CPU / memory spot-check** (Performance) — Headless browser session stability
79. **Authentication testing** (Security) — Login/logout, wrong password, empty password
80. **Authorization testing** (Security) — Protected routes without/with invalid JWT
81. **Session management testing** (Security) — JWT localStorage, 7-day expiry, revoke gap
82. **Token security testing** (Security) — Garbage token, payload tampering rejection
83. **OWASP Top 10 review** (Security) — Mapped findings to OWASP categories
84. **XSS testing** (Security) — Stored/reflected payloads in name, notes, messages
85. **SQL injection testing** (Security) — Parameter fuzzing via ORM usage review
86. **CSRF testing** (Security) — Bearer-token architecture reduces classic CSRF
87. **IDOR / broken access control testing** (Security) — Cross-user booking/ride/profile access
88. **Mass assignment testing** (Security) — Extra fields on ride create/update
89. **Privilege escalation testing** (Security) — Passenger self-confirm, self-verify
90. **Account enumeration testing** (Security) — Login responses for unknown vs wrong password
91. **Brute force / rate limiting testing** (Security) — Burst requests on unprotected routes
92. **Password policy testing** (Security) — Min length 6, complexity expectations
93. **Input validation / fuzzing** (Security) — Invalid email/phone/status/limit params
94. **Security header testing** (Security) — CSP, XFO, HSTS, Permissions-Policy audit
95. **HTTPS / transport testing** (Security) — HSTS header present on live site
96. **CORS testing** (Security) — Evil origin probe on API
97. **Sensitive data exposure testing** (Security) — Phone/PII in public APIs
98. **Error message leakage review** (Security) — Generic vs verbose error strings
99. **Injection in API parameters** (Security) — Query/body fuzzing via script
100. **API testing** (API) — All auth/rides/bookings/payments/messages endpoints
101. **HTTP method testing** (API) — GET/POST/PUT usage per REST design
102. **HTTP status code testing** (API) — 200/201/400/401/403/404/500 behaviour
103. **Content-type / payload testing** (API) — JSON body handling
104. **Response schema / contract testing** (API) — success/message/data shape consistency
105. **Pagination / limit testing** (API) — limit/offset parameters
106. **Database constraint testing** (API) — Sequelize validation min/max/isEmail/is
107. **Data integrity testing** (API) — Seat inventory before/after cancel
108. **Transaction / ACID review** (API) — Missing transaction on booking seat update
109. **Seed data validation** (API) — Demo users/rides from backend boot seed
110. **Migration / sync behavior testing** (API) — sequelize.sync force:true impact
111. **Reliability testing** (Non-functional) — DB wipe on restart, mock fallback
112. **Recovery testing** (Non-functional) — Behaviour after failed booking/payment
113. **Install / deploy config review** (Non-functional) — Procfile, render.yaml, PM2, Caddy
114. **Documentation review** (Non-functional) — README claims vs implemented features
115. **SEO / metadata testing** (Non-functional) — Title, meta tags on live HTML

16. **Dead-link testing** (Non-functional) — Forgot password and nav links
17. **Logging / console error review** (Non-functional) — Browser console during critical pages
18. **Health-check testing** (Non-functional) — GET /api/health
19. **Maintainability observation** (Non-functional) — Code smells reported as QA findings
20. **Process / QA readiness review** (Non-functional) — Absence of automated tests
21. **Configuration security review** (Non-functional) — Hardcoded secrets, CORS allowlist
22. **Session-based exploratory testing** (Exploratory) — Time-boxed exploration notes per module
23. **Monkey / random testing** (Exploratory) — Random invalid params and rapid clicks
124. **Key-tour testing** (Exploratory) — Critical path tours without detailed scripts
125. **Persona-based testing** (Exploratory) — Student passenger vs driver personas
126. **Crowd-style scenario testing** (Exploratory) — Multiple concurrent user roles via API
127. **Data-driven testing** (Exploratory) — Scripted datasets for seats/price/status
128. **Mutation-style input testing** (Exploratory) — Mutating valid requests with invalid fields
129. **Chaos awareness review** (Exploratory) — Backend restart impact (DB wipe)
130. **Accessibility regression spot-check** (Exploratory) — Repeat label/alt audits
131. **Performance budget check** (Exploratory) — Home load timing vs expectations
132. **Cross-session data check** (Exploratory) — Data persistence across browser sessions
133. **Error recovery path testing** (Exploratory) — What user sees after failures

Appendix D — References Helpful in Testing (52)

1. **ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE)** — https://www.iso.org/standard/35736.html
2. **ISO/IEC 25023:2016 SQuaRE — Measurement of system and software product quality** — https://www.iso.org/standard/64774.html
3. **ISTQB Foundation Level Syllabus 2018** — https://www.istqb.org/certifications/foundation-level
4. **ISTQB glossary of testing terms** — https://glossary.istqb.org
5. **OWASP Top 10 (2021)** — https://owasp.org/www-project-top-ten/
6. **OWASP Testing Guide** — https://owasp.org/www-project-web-security-testing-guide/
7. **OWASP XSS Prevention Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
8. **OWASP Broken Access Control (IDOR)** — https://owasp.org/Top10/A01_2021-Broken_Access_Control/
9. **OWASP Mass Assignment / Over-posting** — https://owasp.org/www-community/attacks/Mass_assignment
10. **OWASP SQL Injection Prevention Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
11. **OWASP Authentication Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
12. **OWASP Session Management Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
13. **OWASP API Security Top 10 (2023)** — https://owasp.org/API-Security/editions/2023/en/0x11-t10/
14. **CWE-639: Authorization Bypass Through User-Controlled Key (IDOR)** — https://cwe.mitre.org/data/definitions/639.html
15. **CWE-79: Cross-site Scripting** — https://cwe.mitre.org/data/definitions/79.html
16. **CWE-862: Missing Authorization** — https://cwe.mitre.org/data/definitions/862.html
17. **CWE-863: Incorrect Authorization** — https://cwe.mitre.org/data/definitions/863.html
18. **W3C Web Content Accessibility Guidelines (WCAG) 2.1** — https://www.w3.org/TR/WCAG21/
19. **W3C Web Accessibility Tutorials** — https://www.w3.org/WAI/tutorials/
20. **Mozilla MDN — CORS** — https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
21. **Mozilla MDN — HTTP security headers** — https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
22. **Mozilla MDN — Web Storage API** — https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
23. **Mozilla MDN — Responsive design** — https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
24. **JWT.io — Introduction to JSON Web Tokens** — https://jwt.io/introduction
25. **RFC 7519 — JSON Web Token (JWT)** — https://tools.ietf.org/html/rfc7519
26. **Microsoft Learn — REST API design best practices** — https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
27. **Sequelize documentation — Validations** — https://sequelize.org/docs/v6/core-concepts/validations/
28. **Sequelize dialect-specific things (SQLite/Postgres)** — https://sequelize.org/docs/v6/other-topics/dialect-specific-things/
29. **Express.js security best practices** — https://expressjs.com/en/advanced/best-practice-security.html
30. **Helmet.js** — https://helmetjs.github.io/
31. **express-rate-limit** — https://github.com/express-rate-limit/express-rate-limit
32. **Playwright documentation** — https://playwright.dev/docs/intro
33. **Python requests documentation** — https://requests.readthedocs.io/
34. **Karl Wiegers — Writing effective defect reports** — https://www.processimpact.com/articles/bug_report.html
35. **James Bach — Session-based exploratory testing** — https://www.satisfice.com/blog/archives/278
36. **Michael Bolton — The oracle problem** — https://www.developsense.com/blog/2009/08/the-oracle-problem/
37. **IEEE 829 Test Documentation standard (reference)** — https://standards.ieee.org/ieee/829/4317/
38. **ISTQB Defect / Defect lifecycle** — https://glossary.istqb.org/en_US/defect
39. **Google developers — Speed / PageSpeed Insights** — https://developers.google.com/speed
40. **web.dev — Best Practices** — https://web.dev/learn/
41. **Can I use** — https://caniuse.com/
42. **Caddy documentation** — https://caddyserver.com/docs/
43. **PM2 documentation** — https://pm2.keymetrics.io/docs/usage/quick-start/
44. **React documentation** — https://react.dev/learn/managing-state
45. **Vite documentation — env variables** — https://vitejs.dev/guide/env-and-mode.html
46. **React Router documentation** — https://reactrouter.com/home
47. **NIST SP 800-63B password guidelines** — https://pages.nist.gov/800-63-3/sp800-63b.html
48. **CampusPool live deployment** — https://carpool.33.jugaar.ai
49. **CampusPool API health** — https://carpool.33.jugaar.ai/api/health
50. **CampusPool source (local repo README)** — file:///root/campuspool/README.md
51. **CampusPool DEPLOYMENT guide (local)** — file:///root/campuspool/DEPLOYMENT.md
52. **Course defect-report format reference (Lewis-style)** — https://www.processimpact.com/articles/bug_report.html

Appendix E — Methodology

E.1 Approach

1. **Reconnaissance:** Mapped stack, routes, API surface, deploy topology, demo accounts.
2. **Test design:** Selected 133 techniques from ISTQB/ISO 25010 practice; designed partitions and boundary values for seats, price, ratings, passwords, limits.
3. **Execution — API/security:** Scripted HTTP campaigns on production API; captured status codes and JSON evidence.
4. **Execution — UI:** Playwright journeys (register/login/search/ride/booking/profile/mock pages); screenshots; responsive matrix; a11y audit; cross-browser renders.
5. **Execution — white-box:** Reviewed controllers/models/middleware/configs to confirm root causes.
6. **Curation:** Filtered false positives; assigned severity/priority/oracle/impact.
7. **Reporting:** Assembled this A4 print-ready report.

E.2 Severity Scale

Severity	Definition
Critical	Data loss, complete workflow failure, or financial trust break — release blocking
High	Serious functional/security defect harming users, privacy, or business data
Medium	Functional/QA defect that misleads users or hides failures
Low	Minor usability/config/doc defect — backlog

E.3 Oracle Sources

- README feature claims and UI copy · REST/HTTP status semantics
- OWASP / CWE security standards (authz, XSS, PII, rate limiting)
- Business rules in source (own-ride ban, seat min/max, status enums)
- Common UX expectations (validation messages, dead links, mobile overflow)

E.4 Limitations

- Live demo deployment — backend restart wipes DB (reported as Critical finding).
- Payment gateway simulated — real JazzCash/EasyPaisa/bank flows not testable end-to-end.
- Mobile via viewport emulation (no physical iOS/Android devices in lab).
- Email/OTP flows do not exist — outbound notification testing not possible.

Tester: Tehzeeb
B23110106073
University of Karachi

Course Head: Hussain Saleem
SQET/ST (Software Quality Engineering & Testing)
8 October 2026

— End of Report —

Appendix F - Quality Control (QC) Techniques & Tools Applied (22)

Classic seven QC tools and additional QC/inspection techniques used alongside functional testing techniques (Appendix C).

F.1 Seven Basic QC Tools

Tool	How applied	Output / evidence
Cause-and-Effect (Fishbone / Ishikawa)	Mapped defect clusters (auth, booking, payments, UI) to Man/Machine/Method/Material causes; used to prioritise high-severity areas.	Applied in defect curation and root-cause notes on Critical/High findings.
Check Sheet	Structured checklist per module (auth, rides, bookings, payments, messages, safety, mock pages) with pass/fail/defect counts.	84 findings logged against module check sheets during API/UI campaigns.
Control Chart (conceptual)	Monitored defect discovery rate and severity mix across campaign phases (API, UI, white-box) to detect testing stability.	Severity mix tracked: C1/H24/M52/L7 after curation.
Histogram	Distribution of findings by severity and by source (API vs UI vs code) to show quality hotspots.	Dashboard on page 3 summarises counts; Medium dominates (misleading UX/validation gaps).
Pareto Chart (80/20)	Identified that about 25 Critical+High findings drive the majority of release risk (payments, verification, privacy, booking integrity).	Priority Findings section isolates these for fix-first planning.
Scatter Diagram (conceptual)	Compared finding density vs module complexity (booking/payment logic had highest High/Critical density).	Guided recommendation to harden booking/payment state machines first.
Flowchart	Documented carpool lifecycle: Register, Offer/Search Ride, Book, Confirm, Pay, Complete, Rate; annotated failure points.	Used to locate cascade/missing-flow defects (ride cancel, complete-ride, reviews unreachable).

F.2 Additional QC / Inspection Techniques

Technique	How applied	Output / evidence
Peer / Code Review (inspection)	Formal walkthrough of bookingController, rideController, paymentController, auth middleware, models, deploy configs.	Confirmed mass-assignment risk, missing transactions, force:true sync, fake payments.
Walkthrough of requirements vs UI	Compared README claims and navigation items against implemented routes/API.	Identified mock-only pages (Premium, Earnings, Achievements, Smart Match, Marketplace, Sponsored, Cities) and empty admin folder.
Statistical defect sampling review	Sampled API endpoints proportionally (auth, rides, bookings, payments, messages, safety, impact, users).	Coverage beyond happy-path; drove API observations before curation to 84 findings.
Defect clustering / module heat-map	Grouped findings by module and severity to expose systemic issues.	Booking/payment/verification/privacy clusters = systemic risk themes in exec summary.
Regression re-run (manual)	Re-executed critical journeys after each campaign phase to confirm earlier defects still reproduce on live demo.	Confirmed Critical DB-wipe design defect and High booking/payment issues remain open at report date.
Configuration & deployment audit	Inspected PM2 ecosystem, Caddyfile, Procfile, render.yaml, .env files, PORT defaults.	Found broken deploy manifests, IP CORS allowlist, hardcoded JWT secrets.
Data-integrity spot checks	Read seat inventory before/after cancel and double-cancel via GET /rides/:id.	Proved seat inflation 4 to 6 on double-cancel (High finding).
Error-message quality audit	Reviewed generic vs field-level validation messages on create ride/booking/profile.	Documented opaque Failed to create ... messages hiding model validation.
Privacy / PII exposure audit	Unauthenticated GET /rides/:id and authenticated GET /users/profile/:id inspection.	Driver phone and email/phone PII exposed - High findings.
Accessibility DOM audit	Counted missing alt/labels/unlabelled controls on home and search via Playwright evaluate.	Recorded empty buttons and unlabeled inputs - Medium a11y findings.
Responsive overflow measurement	Compared documentElement.scrollWidth vs clientWidth at 360-1440px.	Confirmed horizontal overflow at common phone widths (360-390px).
Console error monitoring	Captured pageerror/error console entries during critical pages.	No blocking JS errors observed on home/search at test time.
CORS evil-origin probe	Sent Origin https://evil.com to API health; inspected Access-Control-Allow-Origin.	No reflection (ACAO null) - good; still reported IP allowlist hygiene issue from code.
Token tampering probe	Modified JWT payload userId with original signature; requested /auth/profile.	Rejected as Invalid token - signature check works (positive security control noted).

Rate-limit burst probe	Rapid sequential POST /reviews to observe throttling.	No throttle on reviews route (rate-limit gap finding).

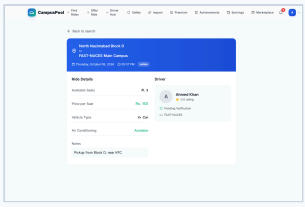
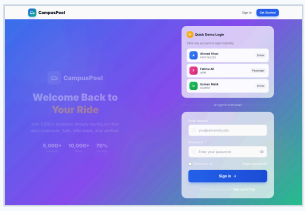
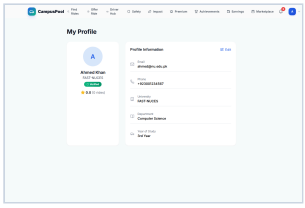
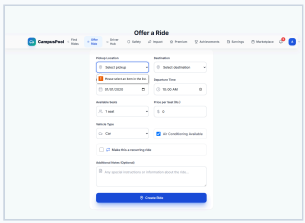
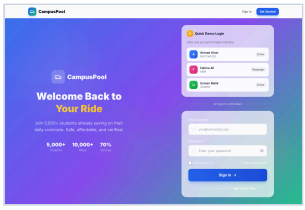
Appendix G - Custom / Innovative Techniques Created (6)

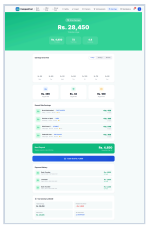
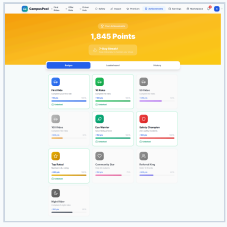
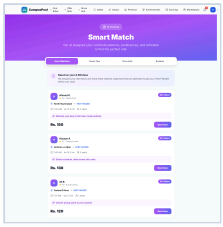
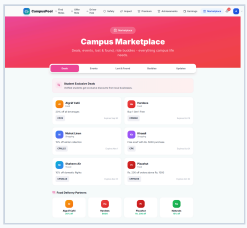
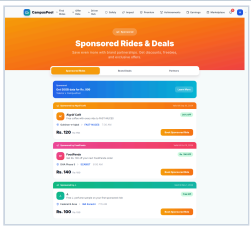
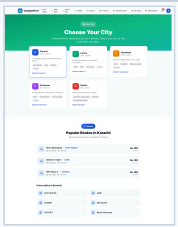
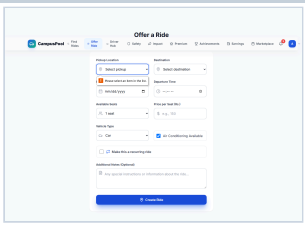
Techniques designed specifically for this campaign beyond standard ISTQB catalogues (brief allows techniques you created).

Custom technique	Description	Why it helped
Demo-DB restart impact analysis	Did not restart live PM2 to avoid wiping shared demo DB; instead reviewed index.ts sync({force:true}) and documented Critical data-loss risk without destructive live test.	Balanced evidence quality vs collateral damage on shared demo environment.
Mock-page authenticity scoring	For each marketing page, scored whether DOM/API traffic hit real /api endpoints or only local React state.	Produced 7 mock-only page findings with consistent oracle: feature must be API-backed or clearly labelled demo.
Verification trust-chain review	Traced Verify button to POST /auth/verify to isVerified flag with no CNIC format/admin check; inspected frontend hardcoded CNIC.	Classified as High trust defect for a student-only carpool platform.
Booking state-machine matrix	Built decision table: roles (driver/passenger/stranger) x actions (confirm/cancel/complete/status) x auth state.	Exposed passenger self-confirm; stranger IDOR blocked correctly; arbitrary status strings accepted.
Search relevance oracle	Entered impossible origin/destination (Lahore to Islamabad) and checked whether Karachi rides still rendered.	Proved free-text filters ignored - functional High finding with UI screenshot.
Severity-Priority crosswalk	Mapped every finding to P1-P4 fix priority aligned with severity and business impact (payments/privacy elevated).	Helped produce clean Priority vs Additional findings split in this report.

Appendix H - Screenshot Evidence Gallery (21 captures)

UI evidence embedded from Playwright runs on the live application. Full-size files also in evidence pack.

Finding	Severity	Screenshot
UI-007 Ride details exposes driver phone publicly	High	 12_ride_details.png
UI-006 Search free-text filter ignored — unrelated rides returned	High	 11_search_lahore_islamabad.png
UI-001 'Forgot password' dead link	Medium	 07_forgot_password.png
UI-015 Student verification fake — hardcoded CNIC / no real check	High	 14_profile_page.png
UI-027 Create Ride past date + 0 seats submit	High	 14c4_create_ride_past_date.png
UI-002 'Remember me' checkbox cosmetic / non-functional	Low	 06_login_page.png
UI-008 Premium page — mock/hardcoded data (no real API)	Medium	 mock_premium.png

UI-009 Earnings page — mock/hardcoded data (no real API)	Medium	 mock_earnings.png
UI-010 Achievements page — mock/hardcoded data (no real API)	Medium	 mock_achievements.png
UI-011 Smart Match page — mock/hardcoded data (no real API)	Medium	 mock_smart_match.png
UI-012 Marketplace page — mock/hardcoded data (no real API)	Medium	 mock_marketplace.png
UI-013 Sponsored page — mock/hardcoded data (no real API)	Medium	 mock_sponsored.png
UI-014 Cities page — mock/hardcoded data (no real API)	Medium	 mock_cities.png
UI-026 Create Ride empty submit validation	Medium	 14c3_create_ride_empty_submit.png

UI-028 Responsive iphone_se 375x667 horizontal overflow	Medium	 resp_iphone_se_home.png
UI-029 Responsive iphone_14 390x844 horizontal overflow	Medium	 resp_iphone_14_home.png
UI-030 Responsive galaxy_s 360x740 horizontal overflow	Medium	 resp_galaxy_s_home.png
UI-032 Responsive desktop_small 1024x768 horizontal overflow	Medium	 resp_desktop_small_home.png
UI-036 Accessibility spot-check home	Medium	 17_a11y_home.png
UI-037 Accessibility spot-check search	Medium	 17b_a11y_search.png
UI-035 Unknown route handling (404)	Low	 16_custom_404.png

Appendix I - Defect Lifecycle & Test Process Followed

I.1 Defect lifecycle used in this campaign

Stage	What was done
1. Detect	API scripts, Playwright UI journeys, and source review discovered candidate issues.
2. Reproduce	Re-ran request/steps; captured HTTP status, JSON body, or screenshot.
3. Classify	Assigned severity (Critical/High/Medium/Low), priority (P1-P4), category, and techniques.
4. Oracle check	Compared against expected behaviour (README, REST semantics, OWASP/CWE, business rules, UX norms).
5. False-positive filter	Removed cases where the app correctly rejected bad input (e.g. auth required, IDOR blocked, seat limit).
6. Report	Logged Lewis-style fields; embedded evidence; listed techniques per finding and in appendices.
7. Recommend	Impact and priority guidance for fix-first planning.

I.2 Test process / levels applied

Level / Phase	Coverage in this campaign
Smoke	Health check, home, login, search baseline
System / E2E	Register, login, search, ride, book, pay, profile, messages journeys
Integration	API + controller + model behaviour via HTTP
Acceptance-style	Business rules: own-ride ban, seat math, status transitions
Non-functional	Security, performance, responsiveness, accessibility, reliability
Static / review	White-box code and config review
Regression	Re-confirmation of critical findings across phases

I.3 Technique count summary

Appendix / Group	Count
Appendix C - Testing techniques catalogue	133
Appendix F - QC tools and inspection techniques	22
Appendix G - Custom techniques created	6
Total distinct techniques / tools applied	**161**

Per-finding technique citations appear on each Priority card and in the Additional Findings register (section 5).